

软件成分分析系统 毕业设计团队总报告

一、团队基本信息

软件成分分析系统团队是由刘峰老师指导的南京大学软件学院 2020 级本科毕业设计团队，团队内各学生成员基本信息如下表。

学号	姓名
201250076	袁家乐
201250084	黄宝俊
201250063	靳琦清
201250074	田姚

二、团队设计背景

自 21 世纪以来，开源软件的发展势头迅猛，全球开源软件项目数量呈指数级增长。根据全球最大开源代码托管平台 GitHub 年度报告数据显示，截至 2023 年 GitHub 平台上总项目已经达到 4.2 亿，公共仓库数量达到 2.84 亿，相比去年增长率超过了 20%。企业在软件开发过程中会选择引入大量开源组件以减少重复工作，提升开发效能。由于开源组件源代码完全公开、贡献者众多、并非封闭管理等特点，开源组件极易被攻击者发现弱点或注入漏洞，引发安全问题。例如 2021 年被称为现代网络安全史上最严重的 Apache Log4j 组件漏洞事件，由于其被应用于各种 Java 程序中，影响极其广泛，全球范围内共涉及 417 万余资产。开源组件的许可证合规性也是不容忽视的内容，盲目使用可能引发知识产权相关的法律纠纷。Google 和 Oracle 两家公司就因为安卓程序的一万余行开源代码存在抄袭嫌疑而深陷十年官司。如何高效地管理项目引入的组件成为软件开发中的重要一环。

软件成分分析（Software Composition Analysis, SCA）应运而生，它通过软件项目包含的一些特征信息，识别分析其中使用的组件、第三方库等，并扫描出其中的漏洞、许可证等信息，以评估软件的安全性、可靠性和可维护性。SCA 一般分为上游和下游两个环节。在上游环节通常关注的是成分识别和依赖分析。根据软件源代码、可执行文件、代码特征文件或 Docker 镜像等软件制品源，分析其中的组件及其依赖关系信息，生成软件物料清单。在下游环节更注重安全隐患管理。一方面开源组件中可能存在已知安全漏洞危害软件产品质量，另一方面开源组件使用的许可证合规性会产生潜在的法律风险。SCA 通过识别归纳软件中存在的漏洞和使用的许可证，针对漏洞给出修复建议，针对许可证明确其规定的权利义务与潜在冲突。由于 SCA 过程并不需要运行目标程序，因此具有方便、快捷、效率高等特点，能有效实现对软件项目的识别、管理、追踪与维护。

如今，开源已被明确列入我国“十四五”规划，要求支持数字技术开源社区等创新联合发展，鼓励企业开放软件源代码、硬件设计和应用服务。同时，我国发布的《网络产品安全漏洞管理规定》也同时指出软件产品中一旦发现安全漏洞，企业应当及时采取措施解决并主动上报。2022 年，中国信通院牵头编制《信息安全技术软件产品开源代码安全评价方法》，可见我国在软件供应链安全技术方面虽然仍处于起步阶段，但国家已经给予高度重视，开源软件和软件供应链安全建设的推进刻不容缓。

根据以上背景，团队与西门子（SIEMENS）公司合作，结合实际业务场景需求，开发了一套软件成分分析系统。该系统主要对软件项目中引入的组件进行快速扫描分析，得出其中的依赖关系，识别其中包含的已知漏洞和使用到的许可证，帮助开发人员尽早发现软件项目中的漏洞隐患和许可证风险，旨在实现软件项目管理的自动化、高效化。

三、团队主要工作与分工

团队从西门子公司实际业务需求出发，深入研究并对比了国内外多款代表性的软件成分分析工具的解决方案（如 OpenSCA、Black Duck、Dependency Track 等）。最终，采用了基于包管理器（构建）的 Build Scan 检测方法，融合西门子公司软件开发规范、组织制度和管理方案，实现了一个综合性的软件成分分析系统。该系统集成了软件制品资产综合管理、依赖信息自动检测和识别、潜在漏洞和许可证合规情况甄别与统计、标准软件物料清单、专业软件成分分析报告生成等多项核心 SCA 功能。

此外，系统整合了多个公开数据源，构建并维护了一个支持增量更新的庞大知识库，包含超过 20 万条 Java、Golang、Python、JavaScript 等主流开发语言的组件详细信息，24 万余条 CVE 和 CNVD 漏洞信息，及 600 余条常用许可证信息。

系统各个模块并非独立，而是互相联系存在交互，并最终形成一个完整的软件成分分析流程。具体的团队分工如下：

1、刘峰：团队指导老师，帮助团队与西门子公司代表人进行需求对接与协商，为系统前期调研、实际开发、最终落地与交付提供了指导性意见。

2、袁家乐：团队队长，负责系统漏洞与许可证分析模块的设计与实现。

3、黄宝俊：团队成员，负责系统 Java、JavaScript 组件依赖关系分析模块的设计与实现。

4、靳琦清：团队成员，负责系统 Python、Go 组件依赖关系分析模块的设计与实现。

5、田姚：团队成员，负责系统应用组件管理、部门用户管理与权限控制模块的设计与实现。

四、系统设计介绍

团队设计的是一套软件成分分析系统，主要面向软件开发和运维人员，用于管理软件制品资产、自动化扫描分析软件制品组件、跟踪漏洞和许可证信息。系统采用前后端分离的 Web 应用架构，客户端发送请求至服务器，服务器通过权限访问控制后返回数据至客户端，由其进行页面更新的操作。数据库选用了 PostgreSQL 和 Redis，PostgreSQL 用于存储系统运行过程中产生的 JSON 对象和长文本数据等复杂类型的数据，Redis 则负责文件传输等情景下的缓存。

系统支持 Java、Python、Go 和 JavaScript 四种语言的组件扫描，实现了知识库管理、组件依赖关系分析、漏洞许可证检测等主要功能，在此基础上增加了团队管理的概念，以部门划分应用，对每位用户赋予角色并授予权限。根据上述功能要点，系统分为了应用组件管理、部门用户管理和权限控制模块、Java、JavaScript 组件依赖关系分析模块、Go、Python 组件依赖关系分析模块、漏洞与许可证分析模块等四个模块。下文将分别针对各个模块进行描述。

（一）项目与组件管理模块

该模块的主要目的是对开发团队创建的应用和组件进行管理，对部门及其成员进行管理，按照部门对应用数据进行隔离，并对用户按照角色进行权限控制。该模块分为应用管理子模块、组件管理子模块、部门管理子模块、用户管理子模块、权限控制子模块。

应用管理子模块提供了一系列针对应用的操作功能。部门产品负责人可以创建应用并指定版本，通过上传应用的特征文件进行软件成分分析，或者对应用子组件进行增减，以管理应用的组件依赖关系。稳定的应用可由部门代表和产品负责人锁定，一旦锁定，将无法对该应用执行任何操作。此外，部门产品负责人能够将应用发布为组件，使其可作为其他应用的子组件使用。产品负责人还可进行应用的版本升级，并实现版本隔离管理。若必要，部门负责人还能删除任何已完全废弃的应用版本。

组件管理子模块负责管理系统的组件库，组件库中的组件有两个来源，系统爬取的开源组件和部门产品负责人上传或者发布应用而成的组件。用户可以查询所有组件的基础信息和依赖关系。部门产品负责人可以修改上传或应用发布成的组件的基础信息或者删除组件。

权限控制子模块实现了基于角色的访问控制，角色的分配由具有相关权限的角色完成。系统管理员具有分配部门代表和部门产品负责人的权限，部门产品负责人有分配应用领导者的权限，应用领导者有分配应用成员的权限。在用户访问系统时，所有前端请求首先会经过身份认证与鉴权层的拦截，此层使用 Spring Security 框架进行用户认证和权限控制。系统通过解析请求头中的 token 来验证用户身份，并从数据库中获取用户权限信息，确保用户具备所请求服务的权限。

（二）Java、JavaScript 组件依赖关系分析模块

该模块的主要目的是对 Java、JavaScript 组件进行依赖关系分析，得到组件的所有依赖及其详细信息，并通过依赖树和依赖平铺表等形式展示组件的依赖关系。该模块分为文件上传子模块、依赖解析子模块、组件爬取子模块和依赖查看子模块。

文件上传子模块负责接收用户上传的代码文件并保存在本地，这些文件是进行后续依赖解析的基础。该模块实现了分片传输和断点续传技术，分片传输允许将大文件拆分为多个小片段并行上传；断点续传则确保在遇到网络问题等情况下，用户无需重新上传整个文件，而是可以从上次中断的位置继续。

依赖解析子模块是软件成分分析系统的关键组成部分，依赖解析为软件的漏洞检测和许可证管理提供了重要支持。模块首先根据上传的文件解析出软件的特征文件的本地路径，然后模块会调用对用的包管理器对特征文件进行依赖解析得到解析结果，接着模块会对包管理器的依赖解析结果进行再解析来提取软件的依赖树信息，然后模块会遍历提取的依赖树并结合系统的组件库和组件爬取功能，得到软件使用的所有开源组件的详细信息并增量更新组件库，最后模块将软件的依赖树信息以及由依赖树生成的依赖表信息存入数据库。

组件爬取子模块是本系统的一个支持模块，不属于主要的业务模块。它的功能是构建开源组件库和在软件依赖解析过程中爬取组件库中缺失的组件。构建的开源组件库可以提高系统的依赖解析效率，也允许用户向应用中增加组件信息。依赖查看子模块主要支持用户查看应用或者组件的依赖树和依赖表信息，并支持应用不同版本之间的依赖树对比。对于查看依赖树，该模块提供了灵活的交互操作，用户可以展开或收起依赖树的各个层级，直观地查看详细的依赖关系。在进行版本对比时，该模块通过使用不同的颜色表示，清晰地显示了新增、更新、删除以及未变的依赖项。

（三）Go、Python 组件依赖关系分析模块

该模块的主要目的是对 Go、Python 组件进行依赖关系分析，得到组件的所有依赖及其详细信息，并支持导出符合 NTIA（美国国家电信与信息管理局）标准的软件物料清单（SBOM）。该模块分为组件依赖关系分析子模块、组件爬取子模块、SBOM 和 Excel 导出子模块。

组件依赖关系分析子模块旨在识别给定的 Go 和 Python 软件中的依赖组件及其关系，这是软件成分分析流程中的关键步骤。系统采用基于包管理器的检测方法，通过识别应用中的特征文件，调用相应语言的包管理器命令获取依赖信息，并解析这些信息生成依赖树。用户进行 Go 语言应用的分析扫描或上传闭源组件时，系统支持上传完整的 zip 项目或 go.mod 文件，系统会确定 go.mod 文件的位置，接着，系统执行 go mod graph 获取依赖信息。在解析 go mod graph 的结果时，通过邻接表将依赖图结构化存储，去除 Go 语言自身的依赖项后，使用广度优先

搜索遍历节点构造依赖树，并去除重复组件。用户进行 Python 语言应用扫描或者上传闭源组件时，系统支持上传 requirements.txt、zip 或 tar.gz 文件，然后系统确定 setup.py 和 requirements.txt 文件的位置。为获取应用依赖信息，系统首先创建虚拟环境以隔离应用的依赖项。然后通过 pip install -r requirements 或者 python setup.py develop 命令安装应用的依赖项。接着，安装 pipdeptree，执行 pipdeptree -json-tree 命令获取 JSON 格式的依赖树信息。在解析依赖树信息之前，系统去除虚拟环境中非项目本身的依赖项。然后递归解析 JSON 格式的依赖树信息，并去除重复依赖。

组件爬取子模块的主要职责是构建开源组件库以及进行依赖解析时查询组件的详细信息。该模块通过使用 HttpClient 从 GitHub 爬取 Go 组件信息，并从 Python 的官方仓库 PyPI 爬取 Python 组件信息。此外，该子模块利用 Jackson 库解析 JSON 格式的数据，以获取各组件的详细信息。

SBOM 和 Excel 导出子模块支持应用级别的 SBOM 和 Excel 导出。导出的 SBOM 按照 CycloneDX 标准，包含应用的所有组件信息，以及这些组件之间的依赖关系。导出的 Excel 包含简要和详尽两种模式，描述了应用所有组件的信息。

（四）漏洞与许可证分析模块

该模块的主要目的是根据上游软件依赖解析的结果结合系统维护的漏洞库和许可证库，获得软件的漏洞和许可证信息，并给出漏洞详情、修复建议和许可证合规分析。该模块分为漏洞分析与管理子模块、许可证分析与管理子模块、统计概览子模块、导出报告子模块。

漏洞分析与管理子模块在预先收集整理了 NVD、CNVD 等公开漏洞数据的支撑上，允许用户检索和查看在管的漏洞的详细信息、上传文件并自动化扫描识别应用或组件的潜在漏洞信息、允许具有权限的用户管理应用的漏洞信息。其中，漏洞信息的自动化扫描依据构造出的组件定位符 CPE 进行数据库本文相似度匹配以锁定对应漏洞。

许可证分析与管理子模块预先收集了 spdx、ScanCode LicenseDB、opensource.org 等公开通用许可证平台的许可证数据信息并基于理解提炼了各种许可证的权利和义务信息。允许用户查看和检索在库的许可证详细信息，使用自动化工具识别应用或组件中使用的许可证和许可证冲突信息，允许具有权限的用户管理应用和组件的许可证信息。许可证的扫描和识别主要是基于带权编辑距离算法，将组件声明的许可证信息与在库许可证做匹配。

统计概览子模块展示部门的软件制品的漏洞和许可证综合情况，整理部门中存在的各风险等级漏洞占比及许可证使用次数排行，分别基于漏洞和许可证对各应用做风险排行。

导出报告子模块主要负责针对指定应用生成详尽的软件成分分析报告，报告中包含了应用及其依赖组件、存在的漏洞、使用的许可证以及冲突的详细描述。

五、总结与评估

本系统是经过严格的市场调研和需求获取之后设计的一款具有完整软件成分分析流程的 SCA 平台。系统采用基于包管理器（构建）的 Build Scan 检测方法，有效支持 Java、Golang、Python、JavaScript 四种语言的组件依赖关系分析，并能够识别与归纳项目中的漏洞风险和许可证合规情况，综合管理软件制品资产，同时自动生成详尽的软件物料清单和综合报告。此外，平台还建立并维护了一个涵盖广泛组件、漏洞和许可证信息的综合知识库。该系统存在以下优点：

1、全流程支持：系统支持了从上游组件依赖关系分析到下游漏洞、许可证分析和报告生成的完整软件成分分析流程，并配备了对应的软件制品资产管理。

2、可扩展性强：系统已支持四种语言的组件依赖关系分析，后续可在小范围调整后支持更多语言。

3、知识库庞大：系统知识库中维护了超过 20 万条组件详细信息、24 万余条漏洞信息、600 余条常用许可证信息。此外，知识库支持增量更新，确保信息的完整性、实时性和准确性。

4、界面清晰友好：系统结合了用户需求和用户体验反馈，不断优化界面设计，提供了可用易用、风格统一、人机交互设计优良的 Web 用户界面。

当然，受团队能力与人力物力资源的影响，本系统仍存在一些不足和可扩展之处。在依赖解析方面，当前系统主要支持基于包管理器引入的依赖解析（build scan），未来可以扩展使用其他检测技术，包括支持对特征文件或者源代码的直接分析（pre-build scan）、基于源码相似度匹配的检测和基于二进制的开源软件识别技术等，并根据不同的需求场景灵活选择。在数据源方面，系统主要依赖于每种语言的官方公共仓库、CNVD 和 NVD 漏洞库以及 SPDX 许可证仓库，未来可集成更多的数据源以提高分析的全面性和准确性。在报告与可视化方面，未来可提供格式更多样、样式更丰富的个性化报告定制和统计图表，以应对不同用户的各种需求。此外，还可以将系统集成到持续集成/持续部署（CI/CD）流程或者软件开发工具中，使软件成分分析更好的融入软件开发生命周期，从而更有效地应对日益复杂的软件安全挑战。